ONE SEMESTER PROJECT

📘 SYSTEM REQUIREMENT SPECIFICS

24/06590

KALOKI CARRICK MWELII

BDS 3106

BACHELOR OF SCIENCE IN DATA SCIENCE

SUPERVISOR - LISPER KENDI

PROJECT TITLE : **SMART BANKLYTICS**

An Intelligent Data – Driven Bank Management and Customer Analytics System

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for **Smart Banklytics**, an intelligent data-driven bank management and customer analytics system. The purpose of this document is to provide a clear and comprehensive description of the system's features, constraints, and operational environment to guide system development and evaluation.

1.2 Scope

Smart Banklytics is a web-based banking system designed to automate banking operations, enhance customer experience, and provide intelligent analytics for bank administrators. The system enables customers to perform secure banking transactions, view and download transaction histories, and manage their accounts. Administrators can manage users, monitor financial activities, and analyze key performance indicators (KPIs) through an integrated analytics dashboard. The system also incorporates a fraud detection mechanism to enhance security and operational reliability.

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
SRS	Software Requirements Specification
KPI	Key Performance Indicator
UI	User Interface
DB	Database
PDF	Portable Document Format

---

2. OVERALL DESCRIPTION

2.1 Product Perspective

Smart Banklytics is a standalone web-based application developed using the Flask web framework in Python. It interacts with an SQLite relational database and provides a user-friendly interface using HTML, CSS, and JavaScript. The system is accessible via modern web browsers.

2.2 Product Functions

The main functions of the system include:

- User authentication and authorization
- Customer account management
- Transaction processing (deposit, withdrawal, transfer)
- Transaction history generation and PDF downloads
- User account suspension and activation
- Analytics dashboard and KPI monitoring
- Fraud detection and transaction monitoring

2.3 User Classes and Characteristics

2.3.1 Customer (User)

Customers are bank clients who interact with the system to perform banking transactions, view balances, access transaction history, download reports, and manage their accounts.

2.3.2 Administrator (Admin)

Administrators are authorized bank staff responsible for managing user accounts, monitoring transactions, controlling system access, viewing analytics, and handling suspicious activities.

2.4 Operating Environment

- Backend: Python (Flask Framework)
- Frontend: HTML, CSS, JavaScript
- Database: SQLite
- Platform: Web browser-based system
- Operating Systems: Windows, Linux, macOS

2.5 Design and Implementation Constraints

- SQLite database storage limitations
- Web-based environment constraints
- Limited computational resources

2.6 Assumptions and Dependencies

- Users have access to internet-enabled devices
- Modern web browsers are used
- The system depends on Python and Flask runtime environment

---

3. SYSTEM FEATURES AND FUNCTIONAL REQUIREMENTS

3.1 User Authentication Module

Description:
Allows users and administrators to securely log into the system.

Functional Requirements:

- The system shall allow registered users to log in using username and password.
- The system shall validate user credentials before granting access.
- The system shall maintain user sessions.
- The system shall redirect users to role-based dashboards.

3.2 Customer Account Management

Description:
Allows customers to manage their personal banking activities.

Functional Requirements:

- The system shall display account details and balances.
- The system shall allow customers to deposit funds.
- The system shall allow customers to withdraw funds.
- The system shall allow customers to transfer money between accounts.
- The system shall display transaction history.
- The system shall allow customers to download transaction history in PDF format.

3.3 Administrator Management Module

Description:
Provides administrative functionalities for system control and monitoring.

Functional Requirements:

- The system shall allow administrators to create user accounts.
- The system shall allow administrators to view and manage all user accounts.
- The system shall allow administrators to suspend or activate user accounts.
- The system shall allow administrators to deposit money into user accounts.

3.4 Transaction Management Module

Description:
Manages all banking transactions and ensures secure processing.

Functional Requirements:

- The system shall record all deposits, withdrawals, and transfers.
- The system shall validate transaction amounts before processing.
- The system shall ensure sufficient balance before withdrawals or transfers.
- The system shall maintain transaction logs for auditing.

3.5 Analytics and KPI Monitoring Module

Description:
Provides analytical insights into banking operations and performance metrics.

Functional Requirements:

- The system shall calculate total users.
- The system shall calculate total transactions.
- The system shall calculate total deposits and withdrawals.
- The system shall calculate total bank balance.
- The system shall display transaction trends using graphical charts.

- The system shall display account type distributions.

3.6 Fraud Detection Module

Description:
Monitors transaction patterns to identify suspicious activities and prevent fraudulent operations.

Functional Requirements:

- The system shall monitor transaction frequency and volume.
- The system shall detect unusually large transactions.
- The system shall detect repeated transactions within a short time.
- The system shall flag suspicious activities.
- The system shall notify administrators of suspicious accounts.
- The system shall automatically suspend accounts involved in potential fraud.

3.7 Password Recovery Module

Description:
Allows secure password reset for users.

Functional Requirements:

- The system shall verify user identity using multiple credentials.
- The system shall allow users to reset forgotten passwords.

---

4. NON-FUNCTIONAL REQUIREMENTS

4.1 Security Requirements

- The system shall provide secure authentication.
- The system shall enforce role-based access control.
- The system shall protect sensitive user data.
- The system shall prevent unauthorized system access.

4.2 Performance Requirements

- The system shall respond to user requests within 3 seconds.
- The system shall support multiple concurrent users.

4.3 Reliability Requirements

- The system shall maintain transaction integrity.
- The system shall prevent data loss during failures.

4.4 Usability Requirements

- The system shall provide a simple and intuitive interface.
- The system shall be accessible via standard web browsers.

4.5 Maintainability Requirements

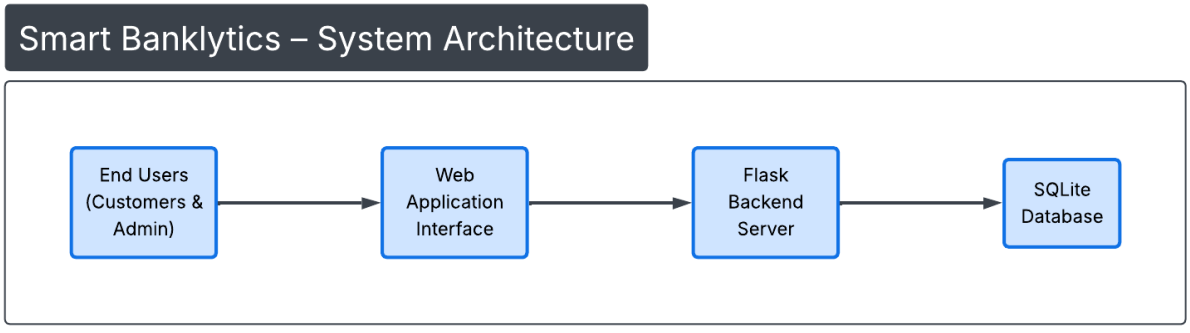
- The system shall be modular and easy to maintain.
- The system shall allow easy upgrades and enhancements.

---

5. SYSTEM ARCHITECTURE

Smart Banklytics follows a **three-tier architecture**:

1. **Presentation Layer** – Web UI (HTML, CSS, JavaScript)
2. **Application Layer** – Flask Backend (Python)
3. **Data Layer** – SQLite Database



6. DATABASE DESIGN OVERVIEW

6.1 Users Table

Stores user account information.

Attributes:

- id
- username
- password
- first_name
- last_name
- id_number
- account_number
- account_type
- balance
- role
- status

6.2 Transactions Table

Stores transaction details.

Attributes:

- id
- account_number
- transaction_type
- amount
- date
- from_account
- to_account

---

7. USE CASE DESCRIPTIONS

7.1 Login Use Case

Actor: User, Admin
Description: Allows registered users to authenticate and access system services.

7.2 Perform Transaction

Actor: User
Description: Allows users to deposit, withdraw, and transfer funds.

7.3 View Analytics Dashboard

Actor: Admin
Description: Allows administrators to monitor KPIs and transaction trends.

7.4 Fraud Detection and Response

Actor: System, Admin
Description: Detects suspicious behavior and automatically flags or suspends accounts.

---

8. CONSTRAINTS AND LIMITATIONS

- SQLite limits scalability for very large datasets.
- Web system requires stable internet connectivity.
- Performance depends on server capacity.

---

9. FUTURE SCALABILITY

- Integration with machine learning-based fraud detection
- Mobile application integration
- Multi-branch banking system support

---

10. CONCLUSION

Smart Banklytics is an intelligent data-driven banking system designed to improve operational efficiency, enhance security, and deliver meaningful insights through analytics and fraud detection. This SRS document provides a complete blueprint for system development, deployment, and evaluation.